The Domain Name System

Lecture 06

Computer Networks and Distributed Programming

Where we are: the step we skipped

Every exchange we have followed so far started after somebody already knew an address.

- A URL carries a name, and no TCP connection can be opened towards a name.
- Delivering mail means finding the machine that accepts it for a domain, which is again a name.
- Each time we said: the address is supplied by the DNS, and moved on.

Today: what answers that question, and what it costs to answer it.

Two ways to name a host

A host can be identified by a hostname or by an IP address, and neither one can be given up.

- People remember `www.unina.it`. Nobody remembers `143.225.161.30`, which is what that name stands for today.
- A name says little about where the host is: it is chosen by an organisation, not by the network.
- An address is fixed in length and hierarchical, which is what makes it usable by routers.
- A name is a variable-length string of characters, which is not.

The two are meant for different readers, so something has to translate between them.

A distributed database, and a protocol

Domain Name System

A distributed database, implemented in a hierarchy of servers, and the application-layer protocol that lets hosts query it.

- It is client-server: a client asks for one translation, a server answers.
- Queries and replies travel in UDP datagrams, on port 53.
- Servers are often UNIX machines running BIND.

It is the one application nobody uses on purpose: it works for other applications, and only shows up when it fails.

Why one server would not do

Suppose one machine held every name-to-address pair in the world.

- Single point of failure: if it crashes, every application that resolves a name stops with it.
- Traffic: it would answer for every Web request and every mail message sent on the planet.
- Distance: it cannot be close to everybody: from the other side of the world every lookup pays the trip.
- Maintenance: one record per host, and hosts are added and moved continuously.

So the DNS is distributed by design: hierarchical, domain-based, and nowhere complete.

The resolver

Resolver

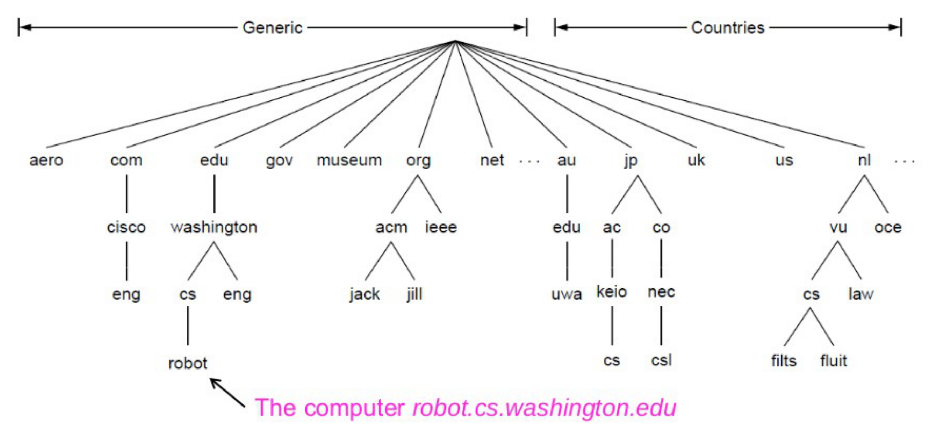
The system component an application calls to obtain the address for a name: it builds the query, sends it, and keeps the answer.

- Your program never speaks DNS: it calls a library function and gets an address back.
- On Linux the stub resolver usually listens on `127.0.0.53` — on your own machine, not out in the network.
- It forwards what it cannot answer to the DNS server configured for the host: the home router, or the provider's.

The name space is a tree

- The naming hierarchy is organised by ICANN, the Internet Corporation for Assigned Names and Numbers.
- There are about 250 top-level domains, of two kinds: generic (`com`, `edu`, `org`, `aero`) and country (`it`, `uk`, `jp`, `tv`).
- Each of them is run by a registrar appointed by ICANN.
- Every domain is partitioned into subdomains, and each of those again, down to the individual host.

The tree



One root, the top-level domains beneath it, everything else further down. The leaf marked here is a host, and its name is the path back to the root.

Generic top-level domains

Seven of them are as old as the DNS itself; the rest were opened in waves from 2001 on.

Restricted means that a registrar has to check that you are entitled to the name — a museum, an air carrier — before selling it to you.

Domain	Intended use	Start date	Restricted?
com	Commercial	1985	No
edu	Educational institutions	1985	Yes
gov	Government	1985	Yes
int	International organizations	1988	Yes
mil	Military	1985	Yes
net	Network providers	1985	No
org	Non-profit organizations	1985	No
aero	Air transport	2001	Yes
biz	Businesses	2001	No
coop	Cooperatives	2001	Yes
info	Informational	2002	No
museum	Museums	2002	Yes
name	People	2002	No
pro	Professionals	2002	Yes
cat	Catalan	2005	Yes
jobs	Employment	2005	Yes
mobi	Mobile devices	2005	Yes
tel	Contact details	2005	Yes
travel	Travel industry	2005	Yes
xxx	Sex industry	2010	No

A name is a path to the root

- A domain name is the sequence of labels from a node up to the root, written leaf first: `cs.keio.ac.jp`.
- An absolute name ends with a dot; a relative one is completed by the resolver before it is sent.
- The same organisation may register under more than one top-level domain, as `ibm.com` and `ibm.us`.
- Cyber-squatting: registering names one has no use for, in order to resell them to whoever will need them.

Who may create a domain

- Each domain controls the allocation of the names immediately beneath it, and nothing else.
- Creating a new domain requires the permission of the parent, which is the only body that can point to it.
- So the tree is not administered from one place: authority is handed down one level at a time.

And the boundaries are administrative, not geographical: two hosts in one domain may sit on different continents, and two machines in the same room may be in different domains.

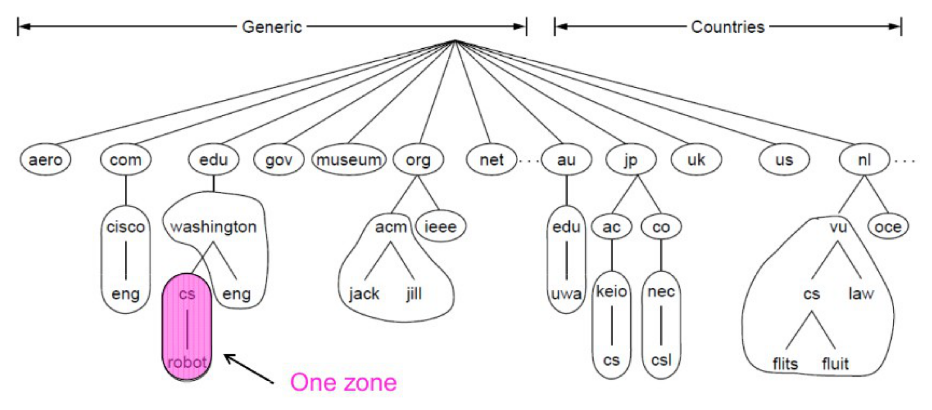
A domain is not a zone

Zone

The part of the tree that one administration actually serves: a domain, minus whatever it has delegated away.

- The name space is divided into non-overlapping zones, and the partition is up to the administrator of each one.
- `unina.it` is a zone; a department given a server of its own becomes a zone of its own, and drops out of the first.
- A domain says what the names look like; a zone says who answers for them.

Zones in the tree



The same tree, cut differently: each outline is a zone, and a zone can be one node or several. No outline ever crosses a delegation.

Primary and secondary

- Every zone has at least two name servers, on separate machines: one primary, one or more secondary.
- Both are authoritative for the zone: their answers are the data itself, not a copy of somebody else's answer.
- The secondary copies the zone from the primary at intervals, and keeps answering when the primary is unreachable.
- An organisation may run them itself or pay a provider to hold the records — most universities run their own.

`a.nic.va` has authority over `vatican.va`: that is where those records actually live.

Three kinds of server in the hierarchy

- Root servers: they know the top-level domains and nothing below: they answer with the servers for `jp`, for `edu`.
- TLD servers: one set per top-level domain, holding the servers of the domains registered under it.
- Authoritative servers: the ones that hold the records of an organisation's own hosts — the bottom of the walk.

The server your host talks to belongs to none of these three: it is outside the hierarchy, and queries it on your behalf.

A resource record

Resource record

One entry of the distributed database. Five fields: name, time to live, class, type, value.

- A DNS reply carries one or more records: they are what a server returns, and what a zone is made of.
- The type is what decides how to read the other two: what the name means, and what the value means.
- The database is not stored anywhere as a whole — these records, spread over many servers, are the database.

Name, time to live, class

- Name: the domain the record applies to. It is the search key of a query, and several records normally exist for the same name.
- TTL: how long a copy of this record may be kept elsewhere: `86400` (a day) for something stable, `60` for something that moves.
- The maximum is $2^{31} - 1$ seconds, about 68 years; the value is chosen by whoever administers the zone.
- Class: `IN`, for Internet. Other classes exist and you will not meet them.

The types

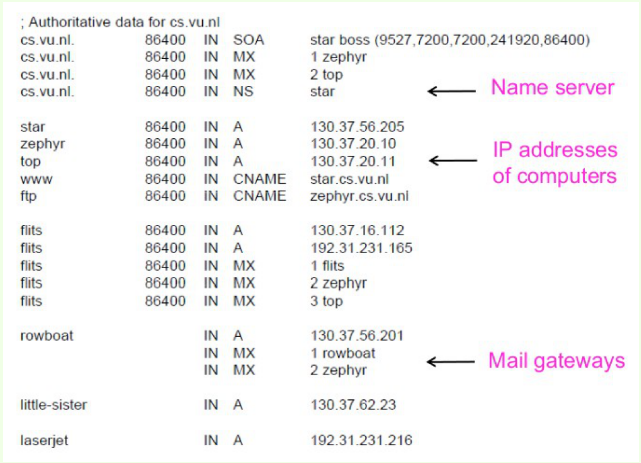
- A: the IPv4 address of an interface of the host; AAAA the IPv6 one. A host may have several of both.
- NS: the name server for a domain. This is the record that carries a query one step further down.
- CNAME: the canonical name behind an alias: the lookup does not stop, it continues with the new name.
- MX: the name of the host that accepts mail for the domain, preceded by a preference value.
- SOA: opens the description of a zone: its main server, its administrator, its parameters.
- PTR: the reverse direction, address to name. Here the lookup does stop, and the name is returned.

Record types

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
SPF	Sender policy framework	Text encoding of mail sending policy
SRV	Service	Host that provides it
TXT	Text	Descriptive ASCII text

The value column is the one to read: for each type it holds a different kind of thing.

Authoritative data for a zone



Every line is one record, in the order name, TTL, class, type, value.

- `NS` says who serves the zone, `A` gives the addresses.
- `www` is not a machine: it is a CNAME onto one.
- Three `MX` with preferences 1, 2, 3: try them in that order.

What the TTL costs you

A name is moved to a new machine: `www.example.it` was `138.12.27.42`, and is now `107.46.72.221`.

- The old pair is not in one place: every server that resolved that name in the last days holds a copy of it.
- Nobody goes and corrects them. Those copies keep answering with the old address until their TTL expires.
- A short TTL propagates a change quickly and costs queries; a long one saves queries and delays the change.

Which is why a TTL is lowered days before a planned move, and raised again once the new address has settled.

Resolving a name

1. The application calls the resolver, which sends the query to the local DNS server configured for the host.
2. If the name falls in a zone that server holds, it answers with an authoritative record, and it ends here.
3. Otherwise it asks the hierarchy on your behalf, starting from the highest thing it already knows about that name.
4. Each server it asks either has the answer, or names a server closer to it.
5. The address travels back to the resolver, and from there to the application that asked.

Recursive from the host, iterative from there on

Recursive query

Whoever is asked must produce the final answer, or an error. The asker waits and does nothing.

Iterative query

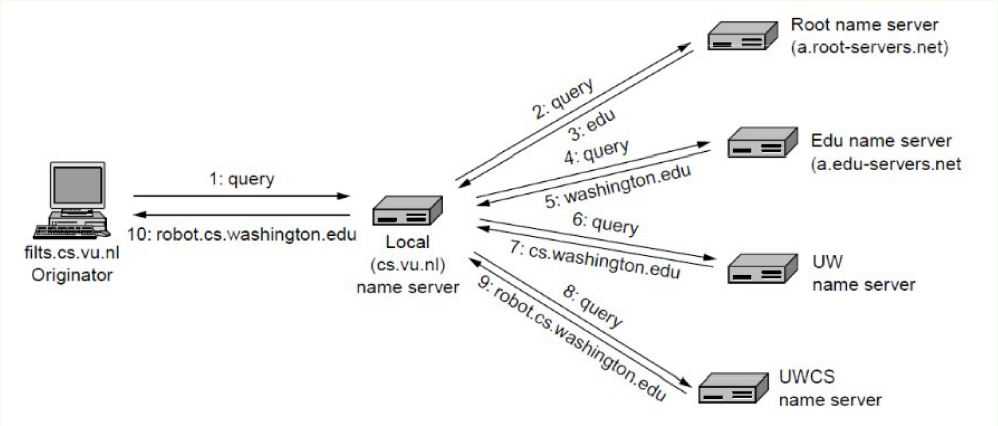
The reply is either the answer or a referral to the next server. The asker keeps working.

- Your host asks once, recursively: one query out, one address back. It never learns that a walk took place.
- The local server does that walk, iteratively, one server at a time.
- Busy servers accept only iterative queries: doing the recursion for everybody would not scale.

The root servers

- There are 13 addresses, `a.root-servers.net` to `m.root-servers.net`, and more than a thousand replicated instances behind them.
- They know the top-level domains, not the hosts: they answer with a referral, never with the address you asked for.
- They are where a resolution starts when nothing about a name is known.
- Thanks to caching, that is rarely the case: most walks start further down.

Ten messages for one name

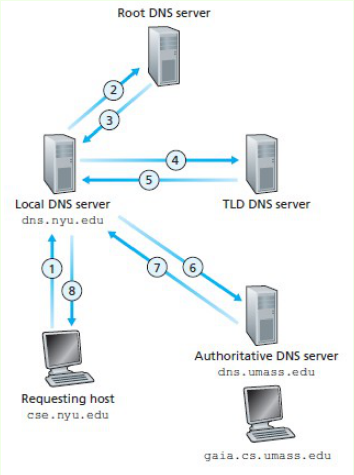


Arrows 1 and 10 are the recursive query of the host; 2 to 9 are the local server walking down the tree, one referral at a time.

The local DNS server

- Run by the ISP or by the institution, and given to the host when it connects.
- It is typically close by — on the same LAN, or a few routers away.
- It belongs to no level of the hierarchy: it acts as a proxy for the hosts it serves.
- Public ones exist too, such as `8.8.8.8` and `8.8.4.4`.

Eight messages here for one name: four queries and four replies.



Caching, and the root you rarely ask

- Every reply a server receives can be cached, and answered from the cache to whoever asks next.
- A cached answer is not authoritative: it comes from a copy, not from the zone that owns the record.
- The addresses of the TLD servers are cached too, so a walk usually starts below the root.
- Not only addresses: a server that knows the name server for a domain can go straight there.

Caching is what makes the whole thing work at this scale — and the TTL is what keeps a cached copy from lying for ever.

dig

```
$ dig vatican.va
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19917
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;vatican.va.                    IN      A

;; ANSWER SECTION:
vatican.va.             21559   IN      A       185.152.70.106

;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
```

- The query went to `127.0.0.53`, the stub resolver on this machine, which forwarded it onwards.
- No `aa` among the flags: not authoritative, the answer came from a cache. `rd`: we asked for recursion; `ra`: the server offers it. The number before `IN` is the TTL.
- What `dig` prints is the DNS message. On Windows you will find `nslookup`, which shows less of it.

Asking for a type

The type goes after the name; with no type, `dig` asks for an address. `+noall +answer` keeps only the answer section.

```
$ dig +noall +answer www.mit.edu A
www.mit.edu.                1070  IN  CNAME  www.mit.edu.edgekey.net.
www.mit.edu.edgekey.net.      51  IN  CNAME  e9566.dscb.akamaiedge.net.
e9566.dscb.akamaiedge.net.    20  IN  A      23.75.122.29
```

Two CNAMEs before an address: the published name is an alias for another name, and the machine that finally answers belongs to somebody else entirely. And look at the TTLs: the alias is kept for minutes, the final address for seconds.

MX returns a name, not an address

```
$ dig +noall +answer mit.edu MX
mit.edu.            59  IN  MX  100 mit-edu.mail.protection.outlook.com.

$ dig +noall +answer mit-edu.mail.protection.outlook.com A
mit-edu.mail.protection.outlook.com.  10  IN  A  52.101.60.185
mit-edu.mail.protection.outlook.com.  10  IN  A  52.101.41.125
mit-edu.mail.protection.outlook.com.  10  IN  A  52.101.9.26
mit-edu.mail.protection.outlook.com.  10  IN  A  52.101.11.19
```

- An MX record gives a name and a preference (100), never an address: a second query is always needed.
- Four addresses for one name: several equivalent mail servers, and mail for this domain is handled by another company.

NS and SOA

```
$ dig +noall +answer mit.edu NS
mit.edu.    840  IN  NS  use2.akam.net.
mit.edu.    840  IN  NS  eur5.akam.net.
mit.edu.    840  IN  NS  asia1.akam.net.     eight of them, on three continents

$ dig +noall +answer +multiline mit.edu SOA
mit.edu.    1800 IN SOA use2.akam.net. network-request.mit.edu. (
                 23589      ; serial
                 900        ; refresh (15 minutes)
                 900        ; retry (15 minutes)
                 3600000    ; expire (5 weeks 6 days 16 hours)
                 3600       ; minimum (1 hour)
                 )
```

- NS lists every server with authority over the zone; SOA names first the single one the others copy from.
- Its parameters are the zone's own clock: how often the secondaries realign, and the minimum TTL of its records.

Being the local server: starting at the root

Ignore the local server and walk the hierarchy by hand, as it would with empty caches. `+norecurse` asks for exactly one step.

```
$ dig +norecurse @c.root-servers.net hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 8, ADDITIONAL: 16

;; AUTHORITY SECTION:
jp.                     172800  IN      NS      a.dns.jp.
jp.                     172800  IN      NS      b.dns.jp.      eight of them

;; ADDITIONAL SECTION:
a.dns.jp.               172800  IN      A       203.119.1.1
b.dns.jp.               172800  IN      A       202.12.30.131
```

The root has no address for the host: the answer section is empty. It delegates instead: the servers of `jp` in the authority section, and their addresses in the additional one — otherwise you would have to resolve their names first. A root server sets `aa` and never `ra`: if the reply carries `ra`, somebody on your line answered for it. L22 comes back to this.

Down to the authoritative server

```
$ dig +norecurse @b.dns.jp hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 3, ADDITIONAL: 3
kyushu-u.ac.jp.            86400   IN   NS   ns2.kyushu-u.ac.jp.
ns2.kyushu-u.ac.jp.        86400   IN   A    133.5.1.9

$ dig +norecurse @ns2.kyushu-u.ac.jp hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 2, ADDITIONAL: 2
ofc.kyushu-u.ac.jp.         3600   IN   NS   ns.jimu.kyushu-u.ac.jp.
ns.jimu.kyushu-u.ac.jp.     3600   IN   A    133.5.40.1

$ dig +norecurse @ns.jimu.kyushu-u.ac.jp hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
hyoka.ofc.kyushu-u.ac.jp.   3600   IN   A    133.5.40.197
```

root → jp → kyushu-u.ac.jp → ofc.kyushu-u.ac.jp: at every step the same question, and a referral to the next server. Only the last reply carries the aa flag: it comes from the zone itself. `dig +trace` does the whole walk in one command — and ends in `BAD REFERRAL` if your line is answering for the root.

Aliasing

- A host whose real name is awkward — `relay1.west-coast.enterprise.com` — is published under a mnemonic alias, `www.enterprise.com`.
- The alias is what people type; the CNAME leads to the canonical name, and the lookup continues from there.
- An application can also ask the DNS the other way round: give me the canonical name behind this alias.
- Mail does the same through MX, so that a company's Web server and mail server can be reached under one name.

The name people know is thus decoupled from the machine that serves them, which can be replaced without anybody noticing.

Load distribution

- A busy site is replicated over many machines, each on its own end system with its own address.
- One canonical name is therefore mapped to a set of addresses, all of them in the database.
- The server replies with the whole set, and rotates the order from one reply to the next.
- Clients normally use the first address they are given, so requests spread across the replicas.

Round-robin DNS costs nothing and needs no new protocol — and it is the same mechanism behind the four mail servers we saw under one name.

What a DNS message carries

Queries and replies have the same format: a 12-byte header, then four sections.

- Header: an identifier copied into the reply, so the client can match answers to questions, plus a set of flags.
- The flags that matter: authoritative answer, recursion desired, recursion available.
- Question: the name being asked about, and the type asked for. Answer: the records that answer it.
- Authority: servers that would know. Additional: helpful records, typically the addresses of the servers just named.

That is all the terminal was showing us: a flag, an authority section, and the addresses in the additional one.

How a record gets into the database

You have registered `example.it`. Nothing in the world can reach it yet.

1. You give the registrar the names and addresses of your primary and secondary name servers.
2. The registrar inserts into the TLD servers an `NS` record for your domain, and an `A` record for each of those servers.
3. Into your own servers you put the `A` record of the Web server and the `MX` record of the mail server.

From that moment the walk down from the root reaches you, and a host on the other side of the world can find your machine without anybody having told it anything.

The answer nobody authoritative sent

We asked the authoritative server itself. Why is the answer not authoritative?

Because whoever replied was not that server. The reply came back without the authority flag, and the flag is set by the machine that holds the zone.

- A DNS query is a UDP datagram to port 53: no connection, no authentication, nothing signed.
- Anything along the path can answer first, in the name of the server you addressed, and the resolver cannot tell.
- Cache poisoning and redirection to somebody else's Web site are the same trick used deliberately.

DNSSEC signs the records so that a forged answer can be detected; encrypted transports hide the query itself. Both are for the end of the course.

Summary

- Names are for people and addresses are for the network: the DNS is the translation, and a distributed database.
- The name space is a tree; a zone is the part of it one administration actually serves.
- Records are typed — `A`, `NS`, `CNAME`, `MX`, `SOA` — and each carries a TTL.
- Your host asks once, recursively; the local server walks the hierarchy iteratively, one referral at a time.
- Caching is what makes this affordable, and the TTL is what bounds how long a stale copy may keep answering.
- What `dig` prints — flags, answer, authority and additional sections — is the message format.

Five questions to a real domain

Pick a domain — `vatican.va`, `unina.it`, one of your own — and ask it five things.

```
$ dig vatican.va                          1.  the address
$ dig vatican.va NS                       2.  who has authority
$ dig vatican.va SOA                      3.  which of them is the main one
$ dig a.nic.va A                          4.  its address
$ dig @212.77.0.110 vatican.va A          5.  ask it directly
```

Steps 4 and 5 use what step 3 returned: the name of the main server, then the address you just resolved for it.

Questions

(a) Which of the five answers come without the `aa` flag, and why is that the expected outcome for the first three?

(b) Does the primary server named in the `SOA` record appear among the `NS` records? Should it?

(c) Which TTL comes back with the first query, and what does it drop to if you repeat the query a minute later?

(d) Step 5 asks the server that holds the zone. Is the answer authoritative? If it is not, who answered?

(e) Try an `MX` query on the same domain. How many queries does it take to reach an address you could open a connection to?